NEOBRIM
Elevating Homes

Software Requirements Specifications (SRS) for AC Class

Requirement #1: Light Class Module

- **Assignee:** Dhanushkumar R
- **Role:** Deep Learning (AI) Intern
- **Date of Assignment:** February 7, 2024
- **Deadline:** February 12, 2024

Home Automation System Implementation

Software Requirements Specification (SRS)

1. AC Class:

Objective:

Design a Python class named AC to represent an air conditioning unit in a home automation system. The class should allow users to control the temperature within a specified range and provide functionalities to fetch the current status.

Specifications:

- **Properties:**
  - **location:** String specifying the location of the AC.
  - **temperature:** Integer representing the temperature setting (ranging from 18 to 30, 0 for OFF).
- **Methods:**
  - **__init__(self, location: str, temperature: int):** Initialize the AC object with given parameters.
  - **control_ac(self, temperature: int):** Control the AC by setting the desired temperature using an IR blaster.
  - **fetch_status(self) -> dict:** Fetch the current status of the AC.

2. Aggregator Class:

Objective:

Develop an Aggregator class in Python to serve as the central controller for the home automation system, focusing on AC control. This class should manage AC objects, handle AI predictions, and save relevant data to a CSV file.

Specifications:

- **Properties:**
    - **time:** Parameter representing the current time.
    - **motion:** Parameter representing the motion status.
    - **fan_status:** Parameter representing the status of the fan.
    - Other relevant parameters.
- **Methods:**
    - **__init__(self, time, motion, fan_status, *args, **kwargs):** Initialize the Aggregator object with given parameters.
    - **define_ac_objects(self, ac_info: List[Dict[str, Union[str, int]]]):** Define objects of the AC class based on provided information, including location and initial temperature settings.
    - **define_ai_object(self, ai_params: dict) -> AIAgent:** Define an object of the AIAgent class with given parameters.
    - **invoke_ac_control(self, ac_obj: AC):** Invoke the relevant control function of the AC class on the provided object.
    - **save_ai_output_to_csv(self, ai_output: str):** Save the AI output in a CSV file (for the last 15 days).
    - **compute_difference(self, previous_state: dict, current_state: dict) -> bool:** Compute the difference between previous and current states. Set manual_override boolean to true if the difference is not null and provide feedback to the AI model.
    - **compute_future_difference(self, current_state: dict, next_predicted_state: dict) -> bool:** Compute the difference between current and predicted states. If the difference is None, do not send a signal.
    - **send_signal(self):** Empty function to send a signal.

3. AIAgent Class:

Objective:

Implement an AIAgent class in Python to serve as an intelligent model predicting the state of AC units. The class should support both Deep Learning and Reinforcement Learning approaches. For Deep Learning, a dummy dataset must be created, and for Reinforcement Learning, a reward function needs to be defined.

Specifications:

- **Properties:**
    - The properties depend on whether it's a Deep Learning or Reinforcement Learning model.
- **Methods:**

- **__init__(self, model_type: str, *args, **kwargs):** Initialize the AIAgent object with the specified model type.
- **If it's a Deep Learning model:**
    - **train(self, dataset: List[Dict[str, Any]]):** Train the model using the provided dataset.
    - **predict(self, input_data: Any) -> Tuple[int, float]:** Predict the state and confidence based on the input data.
- **If it's a Reinforcement Learning model:**
    - **define_reward_function(self, *args, **kwargs):** Define the reward function for the RL model.
    - **train_rl(self, *args, **kwargs):** Train the RL model.
    - **act(self, *args, **kwargs) -> str:** Take an action based on the current state.

Note: Feel free to customize and extend the classes based on your specific project requirements. The provided structure aims to provide a solid foundation for a flexible and extensible home automation system. Adjustments can be made according to the evolving needs and enhancements of the project.